

Análise do Middleware de Monitoramento Prometheus em Sistemas Distribuídos

Rafael Luiz Gonçalves dos Santos
Curso de Sistemas de Informação
Universidade Federal de Uberlândia (UFU) – Campus Monte Carmelo
Monte Carmelo, MG, Brasil
Email: rafael.lui@ufu.br

Abstract—Este artigo apresenta uma análise do Prometheus, um middleware de monitoramento e alerta para Sistemas Distribuídos. Discutem-se os conceitos fundamentais de monitoramento, a arquitetura do Prometheus, sua operação baseada em métricas e um teste prático para demonstrar o funcionamento e a aplicabilidade da ferramenta no contexto de sistemas distribuídos.

Index Terms—Middleware, Sistemas Distribuídos, Prometheus, Monitoramento, Métricas, Grafana.

I. INTRODUÇÃO

Prometheus [?] A transição da arquitetura de software de sistemas monolíticos para ecossistemas distribuídos, baseados em microsserviços, contêineres e computação em nuvem, revolucionou a forma como as aplicações são desenvolvidas e escaladas. Contudo, essa complexidade introduziu novos desafios de observabilidade. O monitoramento tradicional, focado puramente na saúde da infraestrutura — como uso de CPU, memória e disponibilidade de rede —, tornou-se insuficiente para garantir o sucesso e a resiliência de um serviço.

Em um sistema distribuído moderno, uma aplicação pode estar tecnicamente saudável, mas comercialmente inoperante. Considere um cenário prático em um e-commerce: uma atualização no front-end desabilita acidentalmente o botão de "Finalizar Compra". Do ponto de vista da infraestrutura, os servidores estão respondendo, a latência está baixa e não há erros sendo registrados. No entanto, a métrica de negócio mais crucial — o número de vendas por hora — despencou para zero. Sem um sistema que monitore o comportamento do negócio, essa falha crítica poderia passar despercebida por horas, resultando em perdas financeiras significativas.

É para preencher essa lacuna entre o monitoramento de infraestrutura e a inteligência de negócio que ferramentas como o Prometheus [?] se tornaram essenciais. Nascido na SoundCloud e hoje um projeto de código aberto mantido pela Cloud Native Computing Foundation (CNCF), o Prometheus é um toolkit de monitoramento e alerta projetado para a dinâmica e a complexidade dos ambientes modernos. Ele opera com base em um modelo de coleta de métricas multidimensional, um poderoso banco de dados de séries temporais e uma linguagem de consulta flexível

(PromQL), permitindo não apenas a observação de recursos de sistema, mas também a instrumentação da própria lógica de negócio.

Este artigo apresenta uma análise do Prometheus, posicionando-o como um middleware fundamental para a observabilidade em sistemas distribuídos. Serão explorados sua arquitetura, seu ecossistema, os conceitos teóricos que guiam sua operação e, por fim, será detalhado um estudo de caso prático para demonstrar sua implementação e os benefícios tangíveis de adotar uma cultura de monitoramento orientada a dados.

II. FUNDAMENTAÇÃO TEÓRICA

Nesta seção, são abordados os principais conceitos que fundamentam o estudo.

- **Sistemas Distribuídos:** Podem ser definidos como um conjunto de computadores autônomos que se apresentam ao usuário como um sistema único e coerente. Esses sistemas compartilham recursos, dados e responsabilidades, mesmo estando fisicamente separados, comunicando-se geralmente por meio de redes de computadores. A arquitetura distribuída é amplamente utilizada em aplicações modernas, principalmente com o crescimento da computação em nuvem, containers e microsserviços. No entanto, a sua construção apresenta desafios inerentes:

- *Falhas parciais:* Diferente de sistemas centralizados, onde uma falha tende a ser total, nos sistemas distribuídos as falhas podem ocorrer apenas em partes específicas — como em um nó, serviço ou rede —, tornando o diagnóstico e a recuperação mais complexos.
- *Latência de rede:* Como os componentes se comunicam através de uma rede, há sempre algum grau de atraso nas mensagens trocadas, o que pode impactar o desempenho e a sincronização.
- *Consistência de dados:* Garantir que todos os nós tenham uma visão coerente e atualizada dos dados é um dos maiores desafios, levando a modelos como a consistência eventual (*eventual consistency*).

Esses desafios exigem abordagens modernas para observabilidade. Ferramentas de monitoramento,

métricas e logs desempenham um papel essencial, transformando-se em uma fonte rica de informações. Como dita o princípio da engenharia: “quem não mede, não gerencia”.

- **Middleware:** Embora o termo seja tradicionalmente associado a tecnologias como brokers de mensagens, o conceito de middleware como uma camada de software que conecta componentes distintos e abstrai sua complexidade aplica-se diretamente ao Prometheus, que funciona como um **middleware de observabilidade**. Ele se posiciona entre os diversos componentes de um sistema distribuído (aplicações em várias linguagens, bancos de dados, infraestrutura) e as ferramentas de análise e alerta (dashboards, sistemas de notificação), provendo serviços essenciais:

- *Padronização e Abstração:* Ele abstrai a heterogeneidade das fontes de dados. Independentemente da tecnologia interna de um serviço, ele se comunica com o Prometheus através de um padrão unificado: um endpoint HTTP `/metrics` que expõe os dados em formato de texto. As bibliotecas cliente e os *exporters* atuam como adaptadores para essa interface comum.
- *Desacoplamento:* O Prometheus desacopla os produtores de métricas (as aplicações) dos consumidores (Grafana, Alertmanager). Ferramentas de visualização consultam apenas a API do Prometheus via PromQL, sem precisar conhecer os detalhes de cada serviço monitorado. Da mesma forma, uma aplicação pode ser substituída ou atualizada sem impacto no sistema de monitoramento, desde que o “contrato” de métricas seja mantido.

Em essência, o Prometheus fornece a “tubulação” fundamental para a observabilidade, permitindo que todas as partes de uma arquitetura distribuída comuniquem seu estado operacional de forma padronizada e centralizada.

- **Arquitetura de Monitoramento (Push vs. Pull):** A coleta de métricas em sistemas distribuídos segue dois modelos principais.

- **Push:** A aplicação é responsável por enviar ativamente (“empurrar”) seus dados para o serviço de monitoramento. Ferramentas como o Elastic Stack (ELK), com componentes como o Filebeat, utilizam este modelo.
- **Pull:** O sistema de monitoramento busca ativamente (“puxar”) os dados diretamente da aplicação. Este é o modelo adotado pelo Prometheus. A aplicação apenas expõe suas métricas em um endpoint, e o servidor Prometheus periodicamente as coleta.

Esta distinção é fundamental, pois impacta diretamente a configuração e a operação do sistema.

- **Métricas e Séries Temporais:** O modelo de da-

dos do Prometheus é baseado em séries temporais (TSDB), que é um conceito fundamental para seu funcionamento.

Uma **série temporal** é um fluxo de valores carimbados com o tempo (*timestamp*), pertencentes a uma mesma métrica e a um mesmo conjunto de dimensões. Cada série é unicamente identificada por seu **nome de métrica** e um conjunto de rótulos (*labels*), que são pares chave-valor. Por exemplo, a métrica `http_requests_total` pode ser dimensionada com labels para criar múltiplas séries temporais distintas:

- `http_requests_total{method="GET", status="200"}`
- `http_requests_total{method="POST", status="404"}`

Essa estrutura multidimensional é extremamente poderosa, pois permite a filtragem e agregação de dados de forma flexível através da linguagem de consulta PromQL.

Para modelar os dados, o Prometheus define quatro tipos de métricas:

- **Counter (Contador):** É um valor cumulativo que apenas incrementa ou é resetado para zero no reinício de um serviço. É ideal para contar eventos, como o número total de requisições, tarefas concluídas ou erros ocorridos. Seu valor bruto raramente é usado; em vez disso, a função `rate()` do PromQL é aplicada para calcular a taxa de ocorrência por segundo.
- **Gauge (Medidor):** Representa um valor numérico único que pode arbitrariamente subir e descer. É utilizado para medições pontuais, como o consumo atual de memória, a temperatura de um sensor, o número de usuários online ou itens em uma fila.
- **Histogram (Histograma):** É um tipo de métrica complexa usada para amostrar e agregar distribuições de valores, geralmente durações de requisições. Ele agrupa as observações em baldes (*buckets*) configuráveis e expõe contadores para cada um, além da soma e da contagem total das observações. Sua principal vantagem é permitir o cálculo de quantis e percentis (e.g., o 95º ou 99º percentil) de forma agregada no servidor Prometheus, utilizando a função `histogram_quantile()`.
- **Summary (Sumário):** Similar a um histograma, ele também amostra observações. No entanto, ele calcula os quantis do lado do cliente (na própria aplicação) e os expõe diretamente. Embora seja mais simples de consultar, é difícil de agregar os dados de múltiplas instâncias de um serviço, tornando o Histograma a escolha preferida para a maioria dos casos de uso em sistemas distribuídos.

III. OPERAÇÃO DO MIDDLEWARE PROMETHEUS

Nesta seção, discute-se a operação e os componentes do ecossistema Prometheus.

- **Arquitetura Interna:** A arquitetura do Prometheus é composta por módulos que trabalham de forma coordenada para coletar, armazenar e disponibilizar métricas, conforme ilustrado na Figura 1.

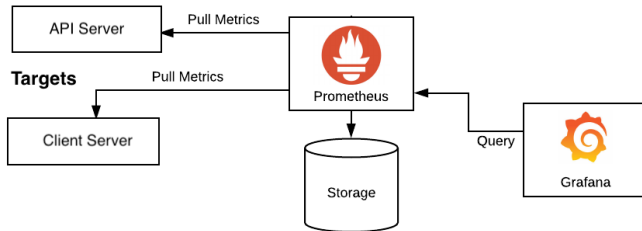


Fig. 1: Arquitetura simplificada do Prometheus, ilustrando o fluxo desde os alvos (*Targets*) até a visualização no Grafana.

Os principais módulos são:

- *Scrape Manager (Coleta)*: Responsável por agendar e executar as requisições HTTP para os endpoints `/metrics` dos alvos.
- *TSDB (Time Series Database)*: Mecanismo de armazenamento otimizado para séries temporais.
- *PromQL Engine*: Módulo que executa as consultas na linguagem PromQL.
- *HTTP Server*: Interface web que expõe os dados e integra com outras ferramentas como o Grafana.
- *Alerting*: Permite a definição de regras de alerta que são enviadas ao Alertmanager.
- **Scrape Manager (Coleta)**: Responsável por agendar e executar as requisições HTTP para os endpoints de métricas (`/metrics`) das aplicações monitoradas. É configurado por meio do arquivo `prometheus.yml`, onde são definidos os *targets* e intervalos de coleta.
- **TSDB (Time Series Database)**: Mecanismo de armazenamento interno otimizado para dados de séries temporais. Cada métrica coletada é armazenada com um timestamp e possíveis rótulos (labels) que permitem consultas avançadas.
- **PromQL Engine**: Módulo responsável por interpretar e executar consultas escritas em PromQL (Prometheus Query Language), que permitem analisar e extrair insights dos dados coletados.
- **HTTP Server**: Interface web embutida que permite acessar as métricas, consultar dados, verificar status dos targets e integrar o Prometheus com outras ferramentas como o Grafana.
- **Alerting Manager (ou integração com Alertmanager)**: Permite a definição de regras de alerta diretamente no Prometheus. Quando uma condição é atendida, a notificação é enviada ao

Alertmanager, que pode repassá-la por email, Slack, webhook, entre outros canais.

• Componentes Principais:

- **Prometheus Server**: O servidor principal que coleta e armazena as métricas.
 - **Client Libraries**: Bibliotecas para instrumentar o código de uma aplicação e expor métricas customizadas.
 - **Exporters**: Programas que expõem métricas de serviços de terceiros (como bancos de dados ou sistemas operacionais).
 - **Alertmanager**: Lida com os alertas enviados pelo Prometheus.
 - **Grafana**: Ferramenta de visualização [?] utilizada para criar dashboards a partir dos dados do Prometheus (embora seja um projeto separado, é o padrão de mercado para visualização).
- **Fluxo de Funcionamento:** O ciclo de operação do Prometheus é um processo contínuo e bem definido, centrado no seu modelo de coleta Pull. Cada etapa do fluxo é crucial para a robustez do sistema.
 - 1) **Descoberta de Alvos (Target Discovery)**: Tudo começa com o Prometheus identificando o que monitorar. Com base no arquivo `prometheus.yml`, ele descobre os alvos através de configurações estáticas (listas de endereços IP) ou, em ambientes dinâmicos, por meio de mecanismos de descoberta de serviços que se integram a plataformas como Kubernetes.
 - 2) **Coleta (Scrape)**: Em intervalos regulares (definidos pelo `scrape_interval`), o servidor realiza uma requisição HTTP GET para o endpoint `/metrics` de cada alvo. Este endpoint, exposto pela aplicação (via biblioteca cliente) ou por um *exporter*, retorna o estado atual de todas as métricas em um formato de texto simples.
 - 3) **Ingestão e Armazenamento**: Ao receber a resposta, o Prometheus realiza a ingestão dos dados. Ele analisa o texto, anexa o carimbo de tempo da coleta a cada métrica e armazena a informação em seu banco de dados de séries temporais (TSDB). Durante este processo, uma métrica sintética chamada `up` é gerada para cada alvo, registrando 1 se a coleta foi bem-sucedida e 0 em caso de falha.
 - 4) **Consulta e Alerta**: Uma vez armazenados, os dados tornam-se imediatamente disponíveis para consulta através da linguagem PromQL. Ferramentas como o Grafana utilizam a API HTTP do Prometheus para executar essas consultas e renderizar dashboards. Simultaneamente, o Prometheus avalia um conjunto de regras predefinidas; se uma regra de alerta for violada pelos novos dados, uma notificação é enviada ao Alertmanager para o devido tratamento.

IV. ESTUDO DE CASO PRÁTICO

Para demonstrar a aplicabilidade do ecossistema Prometheus, foi realizado um teste prático detalhado, desde a instrumentação de uma API até a configuração de dashboards e alertas.

A. Desenvolvimento e Instrumentação da API

O primeiro passo consistiu na preparação do ambiente Python e no desenvolvimento de uma API em Flask que expõe métricas customizadas. As bibliotecas necessárias foram instaladas com os seguintes comandos:

Listing 1: Instalação de dependências Python

```
pip install flask
pip install prometheus-client
pip install prometheus-flask-exporter
```

A API simula o processamento de pedidos e foi instrumentada com um contador para o total de pedidos e um histograma para a latência, conforme o código-fonte apresentado na Listagem ??.

```
1 import time
2 import random
3 from flask import Flask, Response
4 from prometheus_client import Counter,
5   Histogram, Gauge, Summary, Info, Enum
6 from prometheus_flask_exporter import
7   PrometheusMetrics
8
9 # Cria a aplicação Flask
10 app = Flask(__name__)
11 metrics = PrometheusMetrics(app)
12
13 # --- Métricas Personalizadas ---
14
15 # Counter: Total de pedidos processados
16 PEDIDOS_PROCESSADOS = Counter(
17     'meuapp_pedidos_processados_total',
18     'Total de pedidos processados pela
19     aplicação',
20     ['metodo', 'endpoint'])
21
22 # Histogram: Duração do processamento
23 DURACAO_PROCESSAMENTO = Histogram(
24     'meuapp_duracao_processamento_seconds',
25     'Histograma da duração do processamento
26     de um pedido')
27
28 # Gauge: Número de usuários ativos (pode
29     subir e descer)
30 USUARIOS_ATIVOS = Gauge(
31     'meuapp_usuarios_ativos',
32     'Número de usuários ativos na
33     aplicação')
34
35 # Summary: Métrica de latência (parecido
36     com histogram, mas com percentis)
37 LATENCIA_SUMARIO = Summary(
38     'meuapp_latencia_summary_seconds',
39     'Resumo da latência de requisiões')
40
41 # Info: Informações gerais da aplicação
```

```
39 INFO_APP = Info(
40     'meuapp_info',
41     'Informações sobre a aplicação')
42
43 # Enum: Estado da aplicação
44 ESTADO_APP = Enum(
45     'meuapp_estado_aplicacao',
46     'Estado atual da aplicação',
47     states=['inicializando', 'rodando', 'erro'])
48
49
50 # Define informações estáticas
51 INFO_APP.info({'versao': '1.0.0', 'ambiente':
52     'producao'})
53
54 # Define estado inicial
55 ESTADO_APP.state('rodando')
56
57 # --- Endpoints da API ---
58
59 @app.route('/')
60 def index():
61     return "<h1>Olá! Minha API de
62     monitoramento está no ar.</h1>"
63
64 @app.route('/processar_pedido')
65 def processar_pedido():
66     # Simula tempo de processamento
67     aleatorio = random.uniform(0.1, 0.5)
68     # Medindo com Histogram e Summary
69     with DURACAO_PROCESSAMENTO.time():
70         with LATENCIA_SUMARIO.time():
71             time.sleep(tempo_de_processamento)
72
73     # Incrementa contadores e ajusta gauge
74     PEDIDOS_PROCESSADOS.labels(metodo='GET',
75         endpoint='/processar_pedido').inc()
76     USUARIOS_ATIVOS.inc() # Simula aumento
77     de usuários ativos
78     USUARIOS_ATIVOS.dec() # Depois
79     decrementa
80     return f"<p>Pedido processado em {
81         tempo_de_processamento:.2f} segundos.</p>"
82
83 @app.route('/erro')
84 def erro_simulado():
85     ESTADO_APP.state('erro') # Altera estado
86     da aplicação
87     return "<p>Estado da aplicação definido
88     como erro.</p>"
89
90 @app.route('/resetar_estado')
91 def resetar_estado():
92     ESTADO_APP.state('rodando') # Restaura
93     estado
94     return "<p>Estado da aplicação resetado
95     para rodando.</p>"
96
97 # O endpoint /metrics exposto
98     automaticamente
99 if __name__ == '__main__':
100     app.run(host='0.0.0.0', port=5000)
```

Listing 2: API Flask Instrumentada

B. Configuração e Verificação do Prometheus

1) *Instalação e Arquivo de Configuração:* Após o download do Prometheus [?], o arquivo de configuração

`prometheus.yml` foi editado para definir um intervalo de coleta de 15 segundos e adicionar a API Flask como um alvo estático no job `minha_api_flask`. Adicionalmente, uma nova diretiva, `rule_files`, foi adicionada para carregar regras de alerta de um arquivo externo.

Listing 3: Configuração do Prometheus - `prometheus.yml`

```
global:
  scrape_interval: 15s

rule_files:
  - "alert.rules.yml"

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']
  - job_name: 'minha_api_flask'
    static_configs:
      - targets: ['localhost:5000']
```

2) *Execução e Verificação dos Alvos:* A API foi executada utilizando o comando `python app.py` na pasta onde está localizado o script. O servidor Prometheus foi iniciado com o comando `prometheus.exe --config.file=prometheus.yml` dentro da pasta onde o executável do prometheus está localizado. A verificação inicial foi feita acessando a interface web em `http://localhost:9090` e navegando até a aba *Status > Targets*. Conforme a Figura 2, o alvo `minha_api_flask` foi exibido com o estado **UP**, confirmando que a coleta estava funcionando.

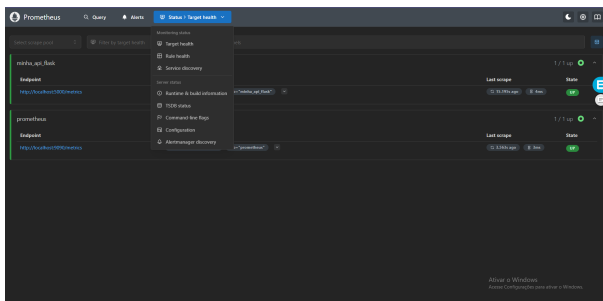


Fig. 2: Tela de Alvos (Targets) do Prometheus, mostrando o job da API com estado UP.

3) *Executando Consultas (PromQL):* Com a API gerando tráfego, diversas queries foram executadas na aba *Graph* do Prometheus para validar a coleta de cada tipo de métrica:

- **Counter:** Para obter a taxa de pedidos por segundo, foi usada a função `rate()`: Figura 3 `rate(meuapp_pedidos_processados_total[1m])`
- **Gauge:** Para ver o número atual de usuários ativos, a consulta é direta: Figura 4] e Figura 5 `meuapp_usuarios_ativos`

- **Histogram:** Para calcular o 95º percentil da latência, a consulta foi: Figura 6 `histogram_quantile(0.95, sum(rate(meuapp_duracao_processamento_seconds_bucket[5m]))by (le))`
- **Summary:** Para obter o mesmo percentil, mas calculado pelo cliente: `meuapp_latencia_summary_seconds{quantile="0.95"}`
- **Info:** Para visualizar as informações estáticas da aplicação: `meuapp_info`
- **Enum:** Para verificar o estado atual da aplicação (retorna 1 para o estado ativo e 0 para os inativos): `meuapp_estado_aplicacao{estado="rodando"}`



Fig. 3: Interface de consulta do Prometheus a taxa de pedidos por segundo

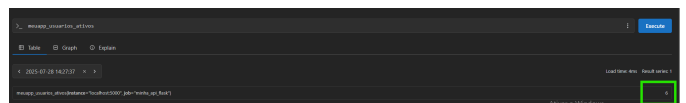


Fig. 4: Interface de consulta do Prometheus exibindo o número atual de usuários ativos tabela



Fig. 5: Interface de consulta do Prometheus exibindo o número atual de usuários ativos grafico



Fig. 6: Interface de consulta do Prometheus exibindo o 95º percentil da latência

4) *Configurando Regras de Alerta:* A configuração de alertas no Prometheus é um processo de duas etapas: primeiro, as regras são definidas em um arquivo YAML dedicado; segundo, o arquivo de configuração principal do Prometheus (`prometheus.yml`) é instruído a carregar essas regras.

a) *Arquivo de Regras de Alerta:* Um arquivo, nomeado aqui `alert.rules.yml`, foi criado para conter a lógica que define quando um alerta deve ser disparado. A Listagem 4 apresenta um exemplo de regra que monitora a disponibilidade dos alvos.

Listing 4: Exemplo de arquivo de regras - `alert.rules.yml`

```
groups:
- name: alertas_basicos
  rules:
  - alert: InstanceDown
    expr: up == 0
    for: 1m
    labels:
      severity: critical
    annotations:
      summary: "Instancia
      {{ $labels.instance
      }} esta fora do ar"

      description: "A instancia {{
      $labels.instance }} do job
      {{ $labels.job }}
      esta inativa ha mais de 1 minuto."
```

A estrutura desta regra é composta pelos seguintes campos:

- **alert:** O nome do alerta (*InstanceDown*).
- **expr:** A expressão em PromQL que define a condição do alerta. A métrica `up` é gerada automaticamente pelo Prometheus, sendo 1 para um alvo saudável e 0 para um alvo que falhou na coleta. Portanto, `up == 0` é a condição para um alvo inativo.
- **for:** A duração de tempo que a condição da **expr** deve ser contínua para o alerta ser disparado. O valor `1m` previne falsos positivos por falhas momentâneas.

- **labels:** Rótulos adicionais anexados ao alerta, usados pelo Alertmanager para roteamento. A label **severity:** `critical` pode ser usada para direcionar este alerta para um canal de emergência.
- **annotations:** Informações detalhadas usadas para construir a mensagem de notificação. Os campos **summary** (resumo) e **description** (descrição) utilizam a sintaxe de template `{{ $labels.nome_do_label }}` para inserir dinamicamente os valores dos rótulos da métrica que originou o alerta, como **instance** e **job**, tornando a notificação clara e informativa.

b) *Vinculando as Regras ao Prometheus:* Para que o Prometheus carregue e avalie essas regras, a diretiva **rule_files** deve ser adicionada ao `prometheus.yml`, conforme mostrado na Listagem 5.

Listing 5: Vinculando arquivo de regras no `prometheus.yml`

```
rule_files:
- "alert.rules.yml"
```

Após a configuração, a regra de alerta pode ser visualizada na aba *Alerts* da interface do Prometheus, como mostra a Figura 7.

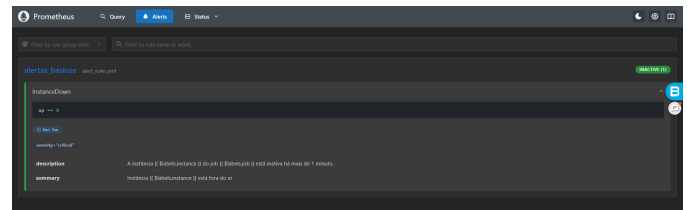


Fig. 7: Interface de visualização dos alertas configurados e gerados

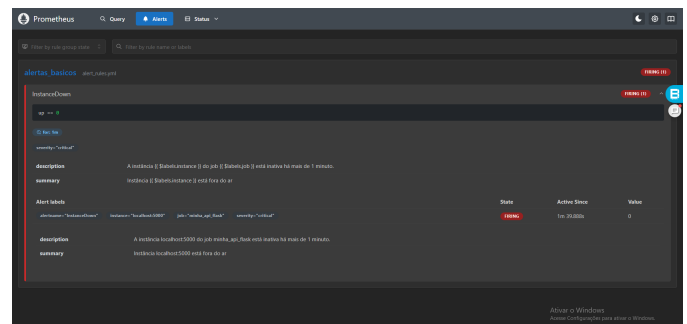


Fig. 8: Teste alerta fora do ar

C. Visualização no Grafana

1) *Conexão com o Prometheus:* Após a instalação do Grafana [?], o primeiro passo foi configurá-lo para buscar dados do Prometheus. Isso foi feito navegando até *Connections > Data Sources*, adicionando uma nova fonte do tipo Prometheus e configurando a URL do servidor como `http://localhost:9090`, conforme

demonstrado na Figura 9. O teste de conexão confirmou que a integração foi bem-sucedida.

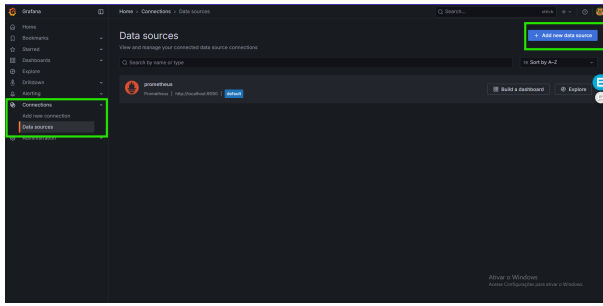


Fig. 9: Configuração do Prometheus como fonte de dados (Data Source) no Grafana.

2) *Criação de um Dashboard*: Um novo dashboard foi criado para visualizar as métricas de negócio da API. Foram adicionados dois painéis principais:

- **Painel de Taxa de Pedidos**: Um gráfico de séries temporais com a query `PromQL rate(meuapp_pedidos_processados_total[1m])`.
- **Painel de Latência P95**: Um painel do tipo *Stat* para exibir o 95º percentil da latência, com a query `histogram_quantile(0.95, sum(rate(meuapp_duracao_processamento_seconds_bucket[5m])) by (le))`.

O resultado final, como visto na Figura 10, é um painel que fornece visibilidade instantânea sobre a performance e a vazão da aplicação.

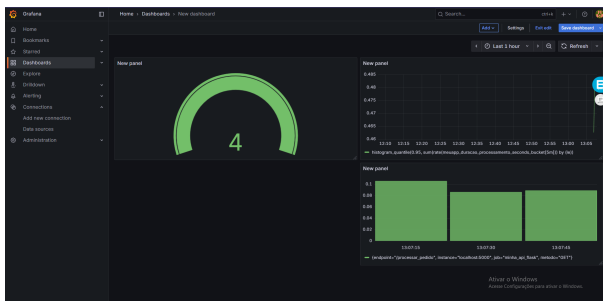


Fig. 10: Dashboard final no Grafana exibindo as métricas de negócio da API em tempo real.

V. CONSIDERAÇÕES FINAIS

O Prometheus se consolidou como uma das ferramentas mais robustas e eficientes para monitoramento de sistemas distribuídos. Sua arquitetura baseada em coleta ativa (*pull*), a poderosa linguagem de consulta PromQL e a integração nativa com diversos serviços o tornam uma solução flexível para a observabilidade moderna.

Durante o desenvolvimento deste estudo, ficou evidente o valor de seus principais tipos de métricas: contadores para o acompanhamento de eventos acumulativos,

medidores para valores voláteis, e histogramas para a análise detalhada de latência. A utilização dessas métricas, expostas automaticamente pelo endpoint `/metrics`, permite diagnósticos rápidos, a definição de alertas precisos e a integração transparente com ferramentas como o Grafana para visualizações em tempo real.

Pessoalmente, o uso do Prometheus proporcionou um melhor entendimento sobre a importância da visibilidade contínua do comportamento de aplicações, principalmente em cenários distribuídos. A ausência de monitoramento adequado nesse contexto pode resultar em falhas silenciosas e degradação da performance, impactando diretamente a experiência do usuário.

Entretanto, algumas limitações foram notadas. O Prometheus, por padrão, não é uma solução ideal para armazenamento de longa duração, o que pode dificultar análises históricas profundas sem o auxílio de outras ferramentas. Além disso, em sistemas com alta efemeridade de instâncias, o gerenciamento de alvos pode se tornar complexo.

Como possíveis trabalhos futuros, destaca-se a integração do Prometheus com bancos de dados externos para armazenamento prolongado, e a sua utilização conjunta com ferramentas de logs (como o Loki) e traces (como o Jaeger), para compor uma solução de observabilidade ainda mais rica, cobrindo os três pilares: métricas, logs e traces.

Em suma, o Prometheus é uma peça fundamental para garantir a confiabilidade e a performance de sistemas distribuídos modernos. Sua adoção capacita as equipes a detectar problemas proativamente e a manter a alta qualidade dos serviços oferecidos.

REFERENCES

- [1] Prometheus Authors, “Prometheus: Monitoring system time series database,” 2025. [Online]. Available: <https://prometheus.io/>
- [2] Grafana Labs, “Grafana: The open observability platform,” 2025. [Online]. Available: <https://grafana.com/>